Raspberry Pi Layer (Software/Raspberry/)

transport/

UartProcessor
uart_processor.py
---
transport: AsyncUartTransport
protocol: DemeterProtocolV2
rx_buffer: bytearray
rx_queue: asyncio.Queue
listeners: List[Callable]
---
start() · stop()
add_listener(cb)
send_command(cmd)
on_uart_data(data)
process_buffer()
_dispatch_loop()
handle_protocol_internal(cmd)
send_ack/nack/syn/ping/set_gpio(...)

owns

uses

AsyncUartTransport
async_uart.py
---
port: str · baud: int
rx_callback: Optional[Callable]
reader / writer: StreamReader/Writer
tx_queue: asyncio.Queue
---
connect() → bool
disconnect()
send(data) → bool
_read_loop() · _write_loop()

implements

«ABC» TransportStrategy
interface.py
---
connect() → bool
disconnect()
send(data: bytes) → bool
set_callback(cb)
start() · stop()

uses

orchestrion/

EdgeServer (FastAPI)
edge_server.py · :8001
---
app: FastAPI
gateway: GatewayOrchestrator
ws_clients: Set[WebSocket]

POST /api/auth/login
GET /api/devices
POST /api/command
WS /ws/{client_id}
broadcast_to_local_clients()
main()

owns

GatewayOrchestrator
command_dispatcher.py
---
uart: UartProcessor
protocol: DemeterProtocolV2
device_manager: DeviceManager
ws_client: DemeterWebsocketClient
---
start() · stop()
dispatch_ws_to_uart(raw_msg)
dispatch_uart_to_ws(cmd)
report_system_config()
_parse_incoming_command(data)

owns

owns

ws_client/ · management/ · utils/

DemeterWebsocketClient
ws_client/client.py
---
uri: str
connection: Optional[WSProtocol]
on_message: Callable
on_open: Optional[Callable]
running: bool
---
start() · stop()
send_json(data)
_connect_loop()

reads config

DeviceManager
management/device_manager.py
---
config_path: str
mappings: Dict[str, Any]
---
load_config() → bool
get_config() → Dict
get_physical_mapping(type, id)
translate_pin(logical_pin)

owns

DatabaseManager
utils/database.py (aiosqlite)
---
db_path: str
---
init_db()
save_reading(node, temp, hum)
get_stats(node, hours) → dict
get_recent_readings(limit) → list

Common Layer (Software/Common/)

DemeterProtocolV2
demeter_protocol.py

_registry: Dict[int, Callable]
---
pack_frame(cmd) → bytes
parse_frame(frame) → Optional[DemeterCommand]
validate_json_message(json) → DemeterCommand
_calculate_crc(data) → int

uses

«enum» CmdId
Nombre	Valor
PING	0x01
ACK	0x02
NACK	0x03
SYN	0x04
SYN_ACK	0x05
SET_GPIO	0x10
SET_PWM	0x11
GET_SENSORS	0x20
EXEC_SEQUENCE	0x30
TEMP_HUM_REPORT	0x08
PIN_REPORT	0x0C
SYSTEM_REPORT	0x0D

parse/pack

Comandos de Control

Ping
type: "ping"

Ack
type: "ack"
original_cmd_id: int

Nack
type: "nack"
original_cmd_id: int
error_code: int

Syn
type: "syn"
context: int

SynAck
type: "syn_ack"
context: int

Comandos de Datos

SetGpio
type: "set_gpio"
pin: int
value: int
flags: int

ExecSequence
type: "exec_sequence"
steps: List[SequenceStep]

SetPwm
type: "set_pwm"
pin: int · value: int

GetSensors
type: "get_sensors"

1..*

SequenceStep
target_id: int
cmd_id: int
pin: int · value: int
delay_ms: int

Reporters (Telemetría)

TempHumReport
type: "temp_hum_report"
node_id: int
temperature: float
humidity: float

PinReport
type: "pin_report"
node_id: int
pin: int · state: int

SystemReport
type: "system_report"
node_id: int
mode: int · battery_mv: int

«abstract» DemeterCommand
BaseModel (Pydantic)
---
target_id: int
source_id: Optional[int]
---
get_cmd_id() → int

Settings
BaseSettings (Pydantic)

PORT: str · UART_BAUD: int
BACKEND_URL · BACKEND_PORT
POSTGRES_* · REDIS_URL
HOST · SOCKET_PORT
LOG_LEVEL · ENV · DEBUG

DATABASE_URL (property)

dispatches